

Serverless Webhook Processing with Retry, Dead-Letter Queue, and Idempotency

Sriraj Gannavaram | Student ID: x23431873

MSc Cloud Computing — National College of Ireland — Short Explanation Guide

Baseline: Qi et al. 2025 Halfmoon (ACM TOCS) — DOI 10.1145/3725985

Artefact: Sriraj/webhook-reliability-eval/ | STATUS: FLOOR COMPLETE (LOCAL_SIM)

1. What is the project?

This MSc research project designs, implements, and evaluates an integrated serverless webhook reliability pipeline. Webhooks are HTTP callbacks used by payments, SaaS products, and CI systems. On AWS they are often ingested through API Gateway and processed by Lambda behind Amazon SQS. Platforms typically provide at-least-once delivery, so duplicates and retries are normal, not exceptional. The floor artefact is Sriraj/webhook-reliability-eval/. It implements four configurations — B, R, RD, and RDI — with HMAC-SHA256 ingestion, exponential backoff plus full jitter, DLQ capture and replay, and a Redis-like idempotency store. Evaluation at floor is a local simulator with green pytest and committed LOCAL_SIM JSON. Live AWS apply is not claimed in the STATUS gate.

One-sentence pitch: I measure how much joint retry, dead-letter queue, and idempotency improve webhook delivery success and duplicate control when faults are injected on purpose at ten, twenty-five, and fifty percent.

2. Problem statement + real-life example

Three failure modes appear together in production webhook paths. Transient network or downstream timeouts discard work if there is no retry. Provider redeliveries and visibility-timeout expiry apply the same business effect twice if there is no idempotency. Poison or schema-invalid payloads either block the main queue or disappear without an audit trail if there is no DLQ. Academic surveys on serverless computing and microservice resilience catalogue these issues separately. They rarely ship one AWS-mapped artefact that ablates retry, DLQ, and idempotency together under a controlled fault grid.

Courier analogy: the driver retries the doorbell with longer waits and random jitter. Parcels that still fail go to a depot holding cage for later replay after the road is fixed. The handheld scanner refuses to mark the same tracking ID delivered twice. HMAC is the tamper-evident seal on the label. A baseline-only consumer is a courier with no seal, no cage, and no scanner — fine on a calm day, expensive on a stormy one.

- Configs (IV): B none → R retry → RD retry+DLQ → RDI full stack.
- Fault rates (IV): 0%, 10%, 25%, 50% as required by CA2.
- Metrics (DV): delivery success rate, duplicate suppression ratio, P50/P95/P99 latency, DLQ recovery rate.

3. Why this matters in industry

Almost every event-driven AWS design that accepts third-party callbacks needs this pattern. Mis-tuned retries without jitter create thundering herds. Retries without idempotency create double charges. DLQs without replay playbooks create silent data loss. This project gives an ablation so a practitioner can see the marginal value of each mechanism, plus Terraform stubs that map to API Gateway, SQS, Lambda, and ElastiCache.

Honesty limits: LOCAL_SIM timings are not CloudWatch p99; cold starts and Redis RTT need live measurement; results do not transfer parameter-for-parameter to Azure Queue Storage or GCP Pub/Sub. STATUS.md records LIVE_AWS=not_run on purpose.

4. Baseline paper — problem, solution, gap, metrics

Primary baseline: Qi, Feng, Liu and Jin (2025), Efficient Fault Tolerance for Stateful Serverless Computing with Asymmetric Logging (Halfmoon), ACM Transactions on Computer Systems, 43(1–2), Article 3, DOI 10.1145/3725985, Online AM 28 March 2025. The OA PDF is stored at [Sriraj/baseline_papers/Qi_et_al_2025_Halfmoon_TOCS_baseline.pdf](https://www.sriraj.net/baseline_papers/Qi_et_al_2025_Halfmoon_TOCS_baseline.pdf).

Problem in Halfmoon: simple retry on stateful serverless functions can corrupt shared state with duplicate updates, while symmetric read/write logging for exactly-once semantics is expensive. Solution: asymmetric logging protocols that log either reads or writes, proven log-optimal, with lower latency and logging overhead than prior systems such as Boki. Metrics emphasise end-to-end latency and logging overhead under function failures.

Gap this CA2 closes: Halfmoon is a runtime logging design. It does not evaluate webhook HMAC ingestion, SQS exponential-backoff with jitter, DLQ capture and replay, or Redis TTL idempotency as a joint application architecture under controlled fault rates. Zhang et al. (2025) TOSEM failure-diagnosis survey (DOI 10.1145/3715005) is kept as related CA2 literature for controlled-fault method language, but its Online AM date in January 2025 sits outside the primary six-to-seven-month recency window used for this pack, so it is not the primary baseline.

5. Research question, hypotheses, and solution

Research question: To what extent does an integrated retry, dead-letter queue, and idempotency framework enhance reliability in serverless webhook processing under faults? RQ1 asks for delivery-success uplift at 10/25/50% faults. RQ2 asks for duplicate suppression under Redis-style keys. RQ3 asks for happy-path latency overhead at P50/P95/P99.

CA2-aligned hypotheses (directional): H1 — RDI raises delivery success far above baseline at 50% fault; H2 — Redis idempotency suppresses $\geq 99\%$ of duplicate deliveries; H3 — DLQ replay recovers a large share of previously failed transient/permanent captures after root-cause fix; H4 — happy-path P50/P95 overhead stays modest relative to

reliability gains.

Solution approach: one harness, four configs, synthetic events, injected transient and permanent faults, duplicate ingestions, DLQ replay after a simulated root-cause fix, JSON metrics. Floor LOCAL_SIM pilot (200 events per cell, duplicate_rate=0.10) shows at 50% fault: B success ≈ 0.57 , R ≈ 0.94 , RD/RDI ≈ 1.00 after replay; RDI duplicate suppression = 1.0; happy-path P50 rises modestly from about 2.5 to 3.2 simulator milliseconds. Direction matches the hypotheses. These are simulator effect directions, not production AWS SLOs.

6. Dataset / data

Synthetic webhook JSON only. Fields: event_id, source, type, amount_cents, currency, poison flag, synthetic=true. Generator: sim.runner.make_event. No personal data and no production customer traffic. Fault mass splits into transient versus permanent (default eighty-five / fifteen of the configured fault rate). Duplicate injection defaults to ten percent. Aggregate metrics live under results/local_sim/ with mode labelled local_sim so they cannot be mistaken for live AWS.

7. Implementation + technologies / AWS services

- Python packages: ingestion (HMAC-SHA256), retry (full-jitter backoff), dlq (maxReceiveCount redrive + replay), idempotency (SET NX EX stand-in), fault injector, pipeline, metrics, sim runner.
- AWS mapping for live phase: API Gateway, Lambda, SQS + DLQ, ElastiCache Redis, Terraform; optional FIS/Locust later per CA2 engineering contribution.
- Floor commands: make setup && make test && make pilot — sixteen pytest cases passed on 2026-09-27 (Asia/Calcutta).
- Terraform stubs currently declare SQS main + DLQ only. Never commit HMAC secrets or AWS access keys.

8. End-to-end flow

- Create synthetic event; HMAC-sign body and timestamp; reject bad signatures with HTTP 401 semantics.
- Enqueue to main queue; optionally enqueue a duplicate event_id for RQ2.
- Worker behaviour by config: B single attempt then drop; R retry until maxReceiveCount; RD/RDI redrive to DLQ; RDI claims an idempotency key — COMPLETE suppresses, PROCESSING may retry.
- Replay DLQ after clearing root cause; count recovered unique event_ids.
- Write pilot_results.json; any future live stack must be destroyed; quote only labelled evidence from STATUS.md.

9. Local pilot numbers you may quote (LOCAL_SIM only)

From results/local_sim/pilot_results.json on 2026-09-27. At zero fault every config reaches success 1.0. At fifty percent fault, baseline B falls to roughly 0.57 success while retry-only

R recovers to about 0.94 and RD/RDI reach 1.0 after DLQ replay. RDI reports duplicate suppression ratio 1.0 whenever duplicates were injected. Happy-path P50 increases only slightly from B to RDI, which answers RQ3 in direction: reliability bookkeeping is not free, but the overhead is small in the simulator. If an examiner asks for CloudWatch evidence, say live AWS has not been run yet and point to STATUS.md.

10. Exact speaking script (dataset + implementation + technologies)

My dataset is synthetic payment-style webhooks — event IDs, amounts, HMAC signatures — with no personal data. I compare four reliability configs: baseline, retry with exponential backoff and jitter, retry plus dead-letter queue, and the full stack with Redis-style idempotency. I inject faults at zero, ten, twenty-five, and fifty percent, plus about ten percent duplicate deliveries. The floor artefact runs locally with in-memory queues and store, sixteen unit tests green, and committed local_sim JSON. Technologies map to API Gateway, Lambda, SQS, ElastiCache, and Terraform, but I do not claim live AWS numbers yet. My baseline is Qi et al. 2025 Halfmoon in ACM TOCS; their gap is runtime asymmetric logging, while I evaluate the joint application mechanisms my CA2 required under controlled fault injection.

11. Five likely Q&A

Q: Is the evaluation live on AWS?

A: No. STATUS says LOCAL_SIM floor; Terraform stubs exist; live apply is future work.

Q: Why Halfmoon instead of Zhang 2025 as primary baseline?

A: Halfmoon is inside the March 2025–September 2026 window and targets FaaS exactly-once; Zhang is a January 2025 diagnosis survey kept as related.

Q: Do you process real customer webhooks?

A: No — synthetic payloads only; ethics scenario is secondary synthetic data on a researcher-controlled evaluation.

Q: What does RDI add over RD?

A: Idempotency keys so duplicate deliveries and DLQ replays cannot double-apply side effects.

Q: Where is the artefact and how do I run it?

A: Sriraj/webhook-reliability-eval/ — run make test and make pilot.

12. Validity, STATUS lock, and honesty rules

Internal validity: shared generator, seeds, and fault model; only reliability mechanisms change across B/R/RD/RDI. External validity: simulator milliseconds omit cold starts and real Redis round-trips. Construct validity: delivery success uses unique event_id sink membership; duplicate suppression counts COMPLETE-key hits against injected duplicate ingestions; DLQ recovery counts unique events restored after replay.

STATUS one-liner for viva: FLOOR COMPLETE — runnable local artefact, sixteen tests passed, Qi et al. 2025 baseline documented, evaluation LOCAL_SIM only (LIVE_AWS=not_run). Never invent AWS dollar costs or CloudWatch percentiles. Never

force-push. Never commit secrets. Prefer creating folders, code, and docs locally; open PRs only when gh authentication allows clean one-commit branches.